SYSTEM ANALYSIS DOCUMENTATION (SAD)

Project Name: InnoGuard AI

Team Name (ID) : Avengers (26)

Domain: AI for Economic Empowerment & Decision Intelligence

System Under Test (SUT) : UMHackathon 2026 - InnoGuard AI

Core Engine: Z.AI General Language Model (GLM)

Version: 1.0

1. Introduction & Purpose 3
1.1. Purpose 3
1.2. Background 3
2. System Architecture & Design 3
2.1. High Level Architecture 3
2.1.1. Overview 3
2.1.2. LLM as Service Layer (Z.AI GLM) 4
2.2. Technological Stack 4
2.3. Key Data Flows 4
2.3.1. Data Flow Diagram (DFD) Summary 4
3. Technical Specifications & Data Flow 6
3.1. Standard Data Flow (Idea Intake to Decision Output) 6
3.2. Model Process (The "Verification Loop" Logic) 6
3.3. Market Explorer (Clustering Logic) 6
4. System Design Elements & Tech Stack 7
5. External Dependencies & APIs 8
6. System Constraints & Assumptions 8
6.1. Technical Constraints * Performance: 8
6.2. Assumptions 8
7. Project Management & Scalability 8

1. Introduction & Purpose

1.1. Purpose

The primary objective of this System Analysis Document (SAD) is to detail the technical framework, architectural choices, and data flows of InnoGuard AI. It serves as a blueprint to ensure the platform can reliably act as a virtual Intellectual Property (IP) strategist, evaluating legal risks and generating actionable business insights for Malaysian SMEs.

1.2. Background

SMEs often face prohibitive costs (RM 10,000+) for preliminary patent research, leading to blind product development. By integrating Z.AI's GLM with a robust vector database, InnoGuard AI automates semantic IP similarity checks and legal document drafting, reducing discovery costs by 90% and time from weeks to minutes.

2. System Architecture & Design

2.1. High Level Architecture

Type	Details
System	Web-based Single Page Application (SPA)
Architecture	FastAPI Backend with Vector-based Semantic Search

2.1.1. Overview

InnoGuard AI is structured as a client-server system where the React/Next.js frontend communicates with a FastAPI backend. The backend coordinates between a Vector Database (FAISS/ChromaDB) for historical patent data and Z.AI's GLM for reasoning and drafting.

2.1.2. LLM as Service Layer (Z.AI GLM)

The GLM acts as the core reasoning engine rather than a generic module.

- **Prompt Construction:** The system expands three simple user prompts (What, How, Who) into a technical parameter set.
- **Context Window:** The context includes the user's idea plus the top 3 similar patents retrieved from the vector database.
- **Response Parsing:** The system parses structured JSON from the GLM to populate the "Risk Dial" and "Reasoning Box".

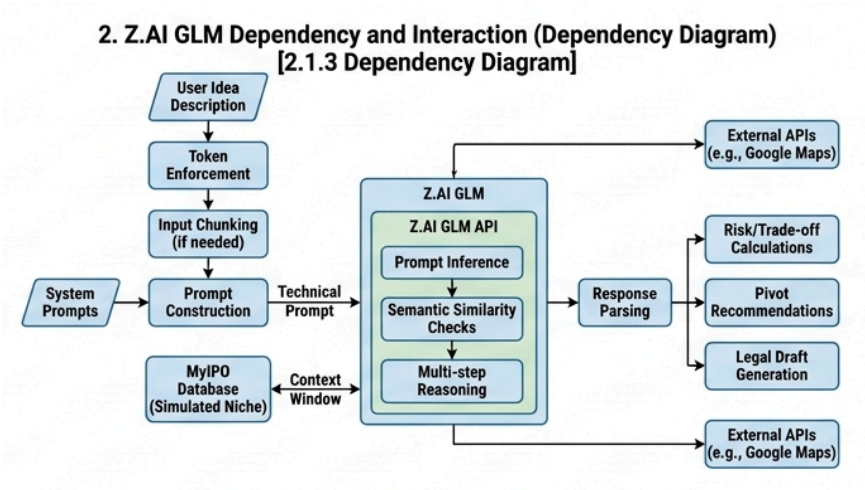
2.2. Technological Stack

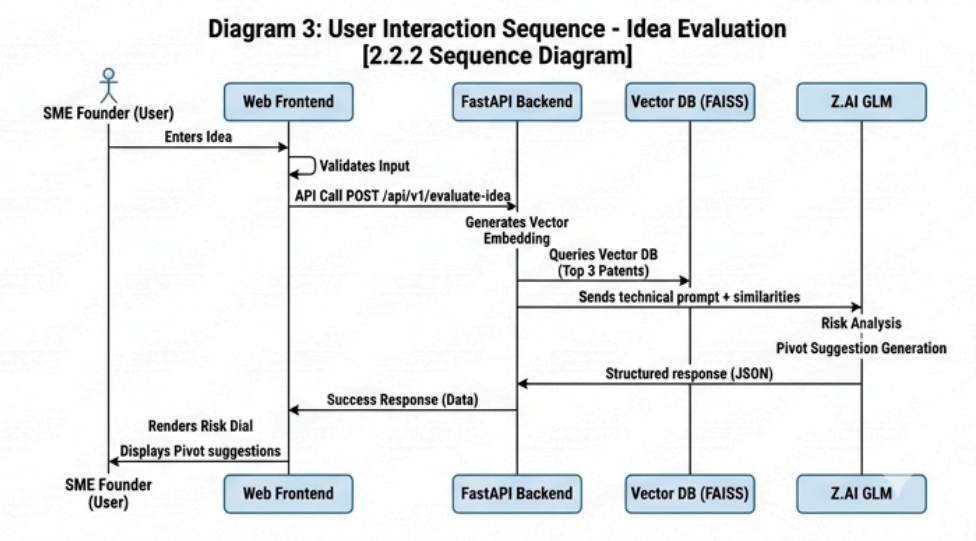
- **Frontend:** React/Next.js, Tailwind CSS, and Shadcn/UI for a "Strategic Command Center" aesthetic.
- **Backend:** FastAPI for high-performance API routing and auto-generated documentation.
- **Database:** FAISS or ChromaDB for vector embeddings; PostgreSQL for user and document persistence.
- **Embeddings:** OpenAI text-embedding-3-small (or Z.AI equivalent) to convert text into 1536-dimensional vectors.

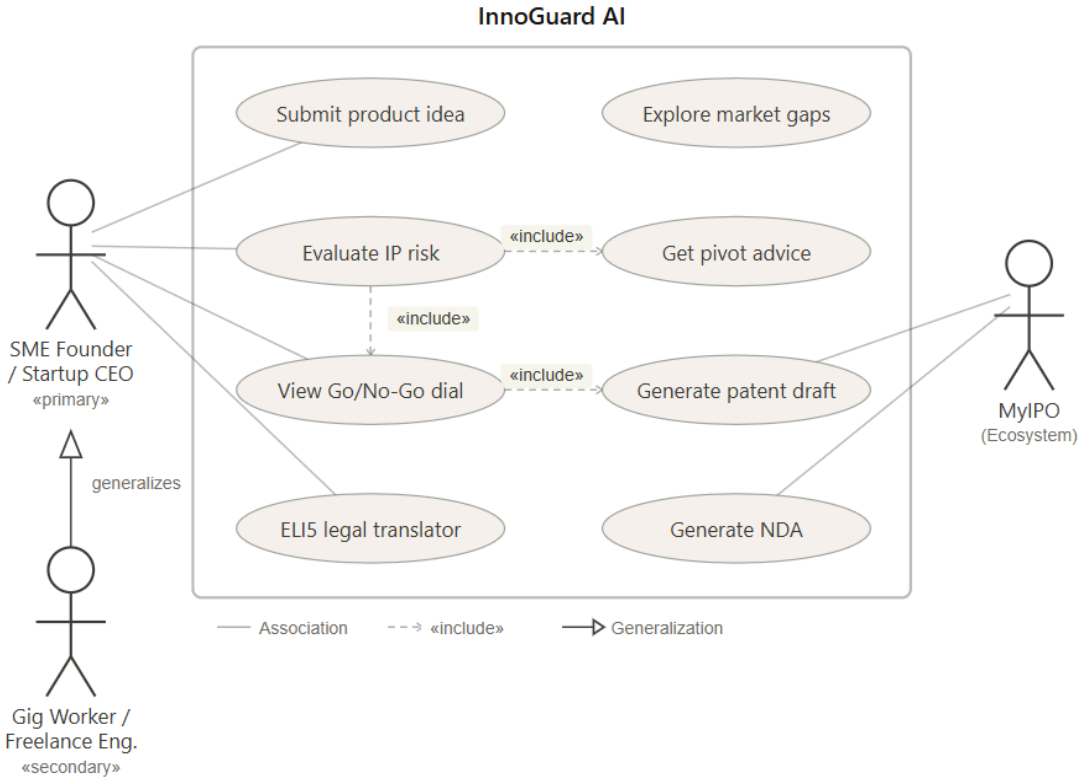
2.3. Key Data Flows

2.3.1. Data Flow Diagram (DFD) Summary

1. **Ingestion:** Scraped MyIPO patent text is converted into vectors and stored in the Vector DB.
2. **Analysis:** User input is converted to a vector → Vector search finds top 3 matches → GLM compares user idea vs. matches → Output generated as risk score and pivot advice.
3. **Finalization:** Selected strategy is fed to the GLM with a MyIPO Form P1 template to generate a downloadable PDF draft.







Use-Case Diagram

3. Technical Specifications & Data Flow

3.1. Standard Data Flow (Idea Intake to Decision Output)

1. **Input:** User submits a product description (What, How, Who) via the Next.js frontend.
2. **Vectorization:** The FastAPI backend converts the unstructured text into a vector embedding.
3. **Retrieval:** The system queries the FAISS database to retrieve the top 3 most semantically similar existing patents (prior art).
4. **GLM Reasoning:** The retrieved patent data and the user's input are fed into Z.AI's GLM. The model calculates a "Safety Score," identifies overlapping technical parameters, and determines the best strategic pivot.
5. **Output:** JSON data is returned to the frontend, updating the Market Explorer's Whitespace Card and the Strategy Lab's Reasoning Box.

3.2. Model Process (The "Verification Loop" Logic)

To prevent hallucinations and ensure high accuracy, the backend implements a secondary safety mechanism:

- **Generate:** GLM outputs a strategic recommendation and draft idea.
- **Verify:** The system internally runs a similarity search on the *newly generated* idea against the vector database.
- **Validate:** If the generated idea triggers a high similarity score (>80%) with an existing patent ID, it is flagged internally and regenerated to ensure the suggested pivot is truly a "Market Whitespace."

3.3. Market Explorer (Clustering Logic)

The platform utilizes **Threshold-based Clustering (K-Means)**. When analyzing thousands of patents within a niche (e.g., Drones), the system groups similar vectors. Areas with low vector density are mathematically identified and presented to the user as recommended "Market Whitespaces" for R&D investment.

4. System Design Elements & Tech Stack

Component	Technology	Justification
Frontend Framework	React / Next.js	Provides server-side rendering capabilities and seamless SPA routing.
Styling & UI Components	Tailwind CSS, Shadcn/UI, Lucide React	Enables rapid development of the "Strategic Command Center" aesthetic with consistent, accessible components.
Backend Framework	FastAPI (Python)	Optimized for high-performance, asynchronous endpoints (/generate-idea, /evaluate-idea).
Vector Database	FAISS / ChromaDB	Required for local, high-speed semantic retrieval of complex engineering/legal terms.
AI/ML Engine	Z.AI GLM	The core decision-intelligence provider; without it, the system reverts to simple, ineffective keyword matching.

5. External Dependencies & APIs

Dependency	Purpose	Failure Handling / Fallback
Z.AI GLM API	Multi-step reasoning and legal drafting.	Implement retry logic and display "Graceful Error State" to the user if the model times out.
MyIPO Data (Simulated)	Source of truth for Malaysian patents.	Use a static, pre-indexed JSON database of "fake" patents for the hackathon simulation to guarantee uptime.

6. System Constraints & Assumptions

6.1. Technical Constraints * Performance:

All decision-intelligence API endpoints must return a response in **< 800ms** to maintain UX.

- **Input Limits:** The system processes a maximum of **1500 tokens** (approx. 800 words). Inputs exceeding this limit are truncated or chunked.
- **UI Constraints:** The frontend relies heavily on specific state animations (e.g., "Thinking Labels" rotating every 2 seconds) to mask AI latency from the user.

6.2. Assumptions

The simulated vector database is structurally identical to real-world MyIPO categorical data.

- The target user has a basic understanding of their own product's mechanics to provide sufficient unstructured data for the semantic engine to process

7. Project Management & Scalability

7.1. Recommendations for Future Scalability * **Vector DB Background Ingestion:** As the patent database scales to millions of entries, vector indexing must be moved to a background cron job to prevent high latency during live searches.

- **Response Caching:** Implement a Redis cache layer for the GLM outputs. If multiple users search for highly similar common ideas, the cached strategic recommendation can be served instantly, drastically reducing API token costs.